# Generative AI

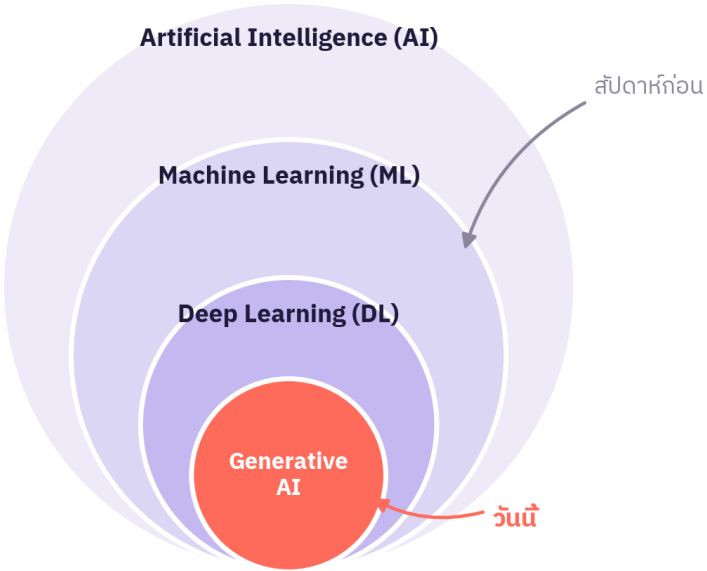
## สถาปัตยกรรม โมเดล และ Large Language Models

ดร.ธีรพงศ์ อรชร



เปิดคาบ

# จากสัปดาห์ก่อน สู่วันนี้




## Machine Learning

เรียนรู้รูปแบบจากข้อมูล แทนการเขียนกฎเอง

## Deep Learning

Neural network หลายชั้น เรียนรู้ feature เองได้

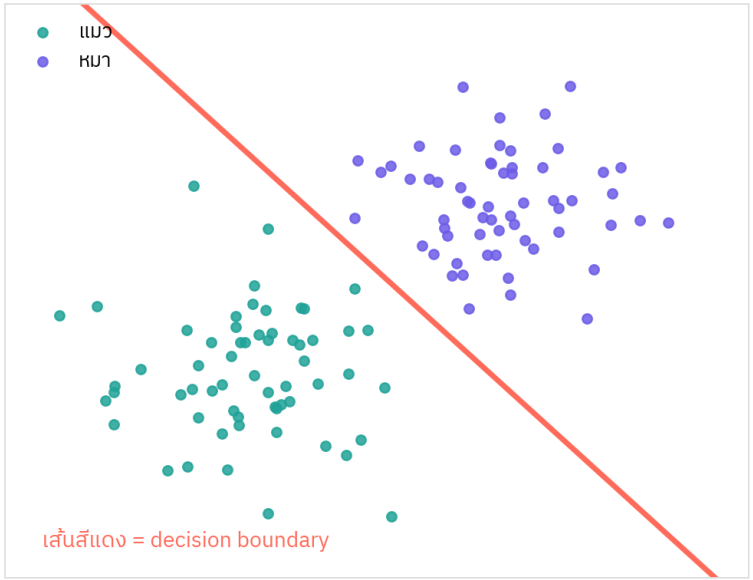
## Generative AI

Deep learning ที่ "สร้างข้อมูลใหม่" ได้

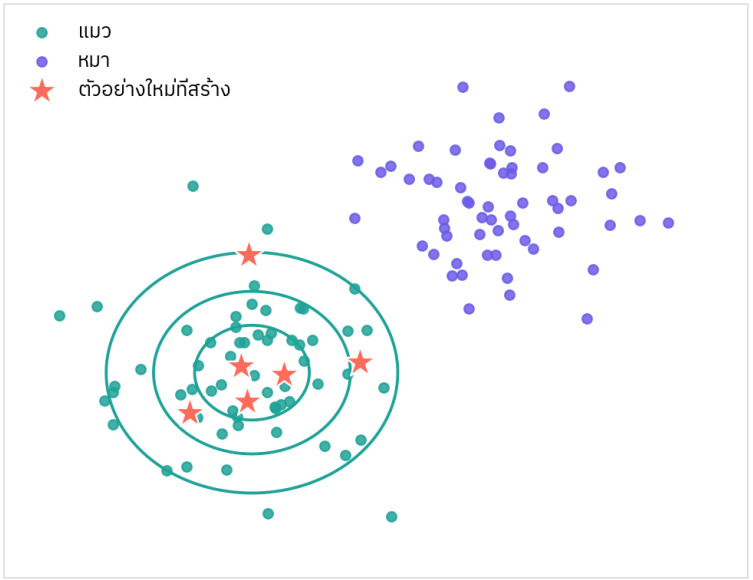
เปิดคาบ

# Discriminative vs Generative

Discriminative: เรียนรู้ "เส้นแบ่ง"
P(y | x) → ตอบว่า "นีคือแมวหรือหมา"




Generative: เรียนรู้ "การกระจายของข้อมูล"
P(x) → "วาดแมวตัวใหม่ให้หน่อย"




โมเดลที่เรียนสัปดาห์ก่อน (เช่น CNN จำแนกภาพ) เป็นแบบ Discriminative — วันนี้เราจะเรียนแบบ Generative

เปิดคาบ

# โจทย์: ให้ AI สรุปผลตรวจคนไข้เป็นภาษาที่อ่านง่าย

**สถานการณ์**

ทีมโรงพยาบาลอยากใช้ Generative AI สรุปผลตรวจเลือด เอ็กซเรย์ และบันทึกแพทย์ ให้คนไข้อ่านเข้าใจได้ง่าย

**A**

**ทำได้เลย**
AI เก่งเรื่องสรุปความอยู่แล้ว

**B**

**ทำได้ แต่ต้องมีคนตรวจ**
ต้องมีแพทย์ทวนก่อนส่งให้คนไข้

**C**

**ยังไม่ควรทำ**
ความเสี่ยงสูงเกินไป

ให้นักศึกษาโหวต แล้วเก็บคำตอบไว้ — เราจะกลับมาดูอีกครั้งตอนเรื่อง Hallucination

เปิดคาบ

# เส้นทางของวันนี้: 5 ส่วน

1. **Generative AI คืออะไร** สร้างอะไรได้ ใช้ที่ไหน
2. **โมเดลสร้างภาพ** VAE, GAN, Diffusion สร้างภาพใหม่ได้อย่างไร
3. **โมเดลสร้างข้อความ** จาก RNN → Transformer → LLM → ChatGPT
4. **Hallucination** ทำไม LLM ตอบผิดอย่างมั่นใจ แก้อย่างไร
5. **เครื่องมือ** ของที่เราจะใช้ใน Colab บ่ายนี้

เส้นเรื่อง: ส่วน 2–3 ตอบว่า "ข้างในทำงานอย่างไร" · ส่วน 4 ตอบว่า "ระวังอะไร" · บ่ายนี้ลงมือทำจริง

# นิยาม Generative AI

*"Deep learning models ที่สร้างเนื้อหาคุณภาพสูง เช่น ข้อความ ภาพ และเนื้อหาอื่นๆ จากข้อมูลที่ใช้ฝึก"*

**① เรียนรู้**
ดูข้อมูลจำนวนมาก
(ภาพ ข้อความ เสียง)

→

**② จับรูปแบบ**
เข้าใจโครงสร้างและ
รูปแบบที่ซ่อนอยู่

→

**③ สร้างใหม่**
ผลิตเนื้อหาใหม่
ที่ไม่ซ้ำของเดิม

เปรียบเทียบ: เหมือนศิลปินที่ดูภาพวาดมามากมาย จับสไตล์ได้ แล้ววาดผลงานใหม่ของตัวเอง

# Generative AI สร้างอะไรได้บ้าง

| ข้อความ | ภาพ | เสียง | 3 มิติ | ดนตรี |
|---|---|---|---|---|
| Text | Image | Audio | 3D Objects | Music |
| GPT, Claude | DALL-E, GAN, Diffusion | WaveNet, Text-to-Speech | Text-to-3D | Music generation |

วันนี้เจาะลึก 2 กลุ่ม: ภาพ (ส่วนที่ 2) และข้อความ (ส่วนที่ 3) — ซึ่งนำไปสู่ LLM

# การสร้างข้อความ (Text Generation)

- เข้าใจบริบทและความสัมพันธ์ระหว่างคำ/วลี
- สร้างข้อความที่สอดคล้องกับบริบท
- ตัวอย่างโมเดล: GPT (Generative Pre-trained Transformer)

### แต่งเรื่องต่อ

Input: "หุ่นยนต์ตัวหนึ่งตื่นขึ้นมากลางดึก..."

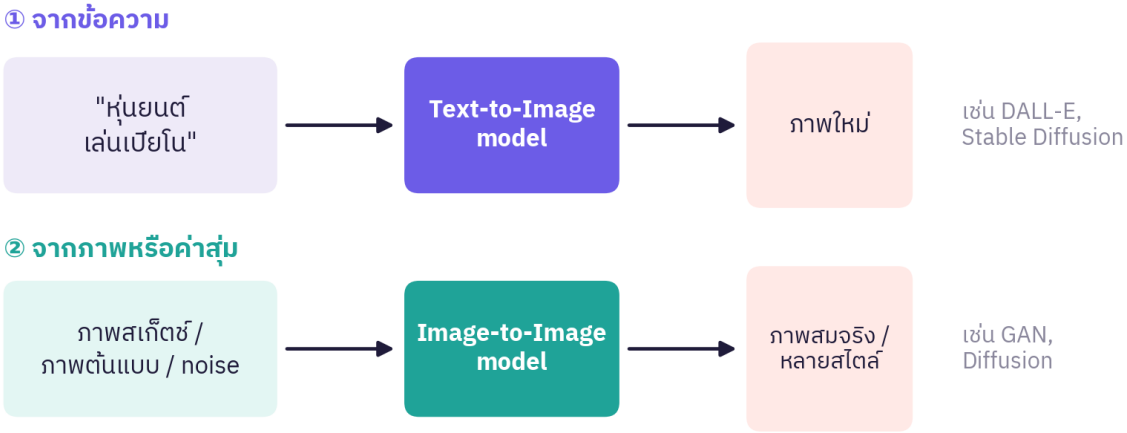
Output: "...มันได้ยินเสียงเปียโนจากห้องข้างๆ"

### แปลภาษา (คงน้ำเสียงเดิม)

"Could you kindly send me the file?"

→ "รบกวนส่งไฟล์ให้หน่อยได้ไหมคะ"

# การสร้างภาพ (Image Generation)




**รู้หรือไม่?**

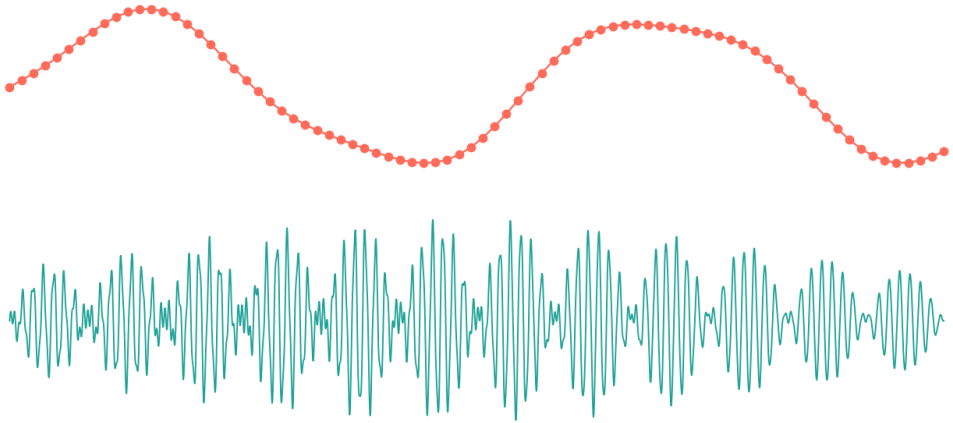
ชื่อ DALL-E มาจาก Salvador Dalí + WALL-E (ไม่ได้เป็นตัวย่อ)

**ข้อควรระวัง**

Deepfake: ภาพ/วิดีโอปลอมที่สมจริง ใช้หลอกลวงได้

# การสร้างเสียง (Audio Generation)

ซูมเข้าไป: WaveNet ทำนายเสียงทีละ "จุด" (sample) ต่อเนื่องกันหลายหมื่นจุดต่อวินาที



- สร้างเสียงพูดที่ฟังเป็นธรรมชาติ
- Text-to-Speech: แปลงข้อความเป็นเสียง
- ตัวอย่างโมเดล: WaveNet

**การใช้งาน: ผู้ช่วยเสียง หนังสือเสียง พากย์วิดีโอ ระบบตอบรับอัตโนมัติ**

# Generative AI ถูกใช้ที่ไหนบ้าง

**$1.3T**

มูลค่าตลาดที่คาดการณ์ภายในปี 2032

ที่มา: Bloomberg Intelligence

### งานทั่วไป

สร้างคอนเทนต์ · สรุปเอกสาร · แปลภาษา · Chatbot

### การแพทย์

วิเคราะห์ภาพทางการแพทย์ เขียนรายงานคนไข้

### การเงิน

พยากรณ์จากข้อมูลการเงินขนาดใหญ่

### เกม

เนื้อเรื่องและตัวละครที่เปลี่ยนตามผู้เล่น

### IT / Data

สร้าง synthetic data มาเทรนโมเดล

ส่วนที่ 2

# โมเดลสร้างภาพ: VAE · GAN · Diffusion

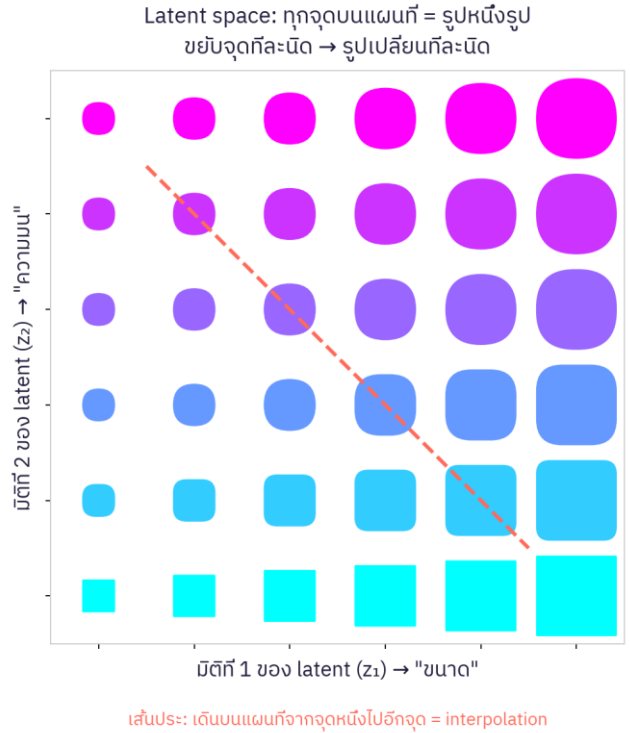
**จากส่วนที่แล้ว:** เรารู้แล้วว่า GenAI สร้างข้อความ ภาพ เสียงได้ ต่อไปดูว่า "ข้างใน" ทำงานอย่างไร เริ่มจากภาพเพราะมองเห็นง่ายที่สุด

**คำถามของส่วนนี้**

ถ้าเราสอนให้คอมพิวเตอร์ "รู้ว่าภาพแมวหน้าตาแบบไหน" แล้ว เราจะให้มันวาดภาพแมวตัวใหม่ได้อย่างไร?

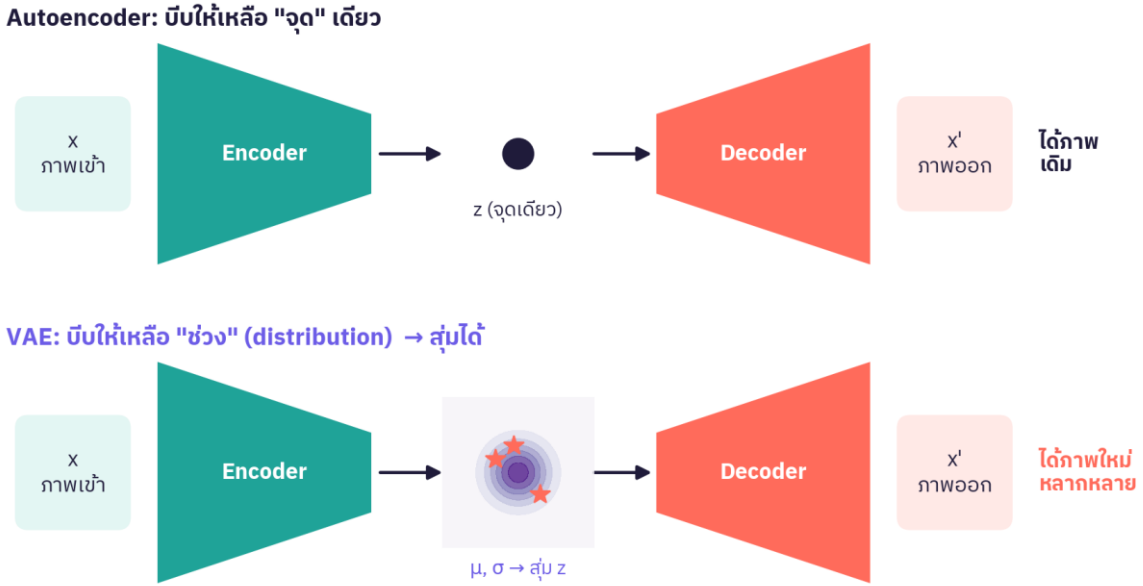
(โจทย์ท้ายวัน: ถ้าต้องสร้าง "วิดีโอจากข้อความ" จะใช้โมเดลอะไร)

# Latent Space: "แผนที่" ของข้อมูล



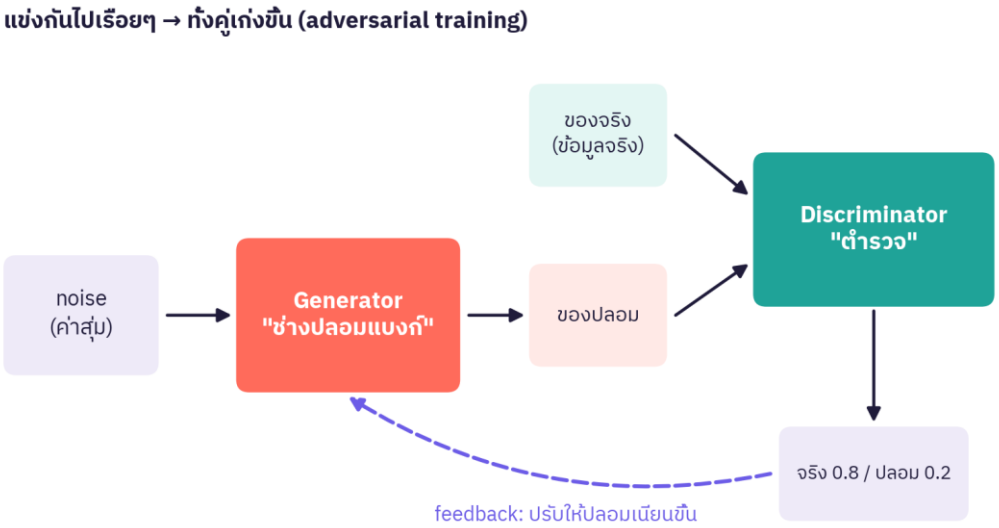

- ภาพ 1 ภาพมีหลายแสนพิกเซล แต่ "สาระ" ของภาพอธิบายได้ด้วยตัวเลขไม่กี่ตัว
- Latent space = พื้นที่ของตัวเลขชุดย่อนั้น
- จุดที่อยู่ใกล้กัน = ข้อมูลที่หน้าตาคล้ายกัน
- **หยิบจุดใหม่บนแผนที่ → ได้ข้อมูลใหม่ = หัวใจของการ "สร้าง"**
- VAE, GAN, Diffusion ล้วนใช้แนวคิดนี้

# Autoencoder → VAE



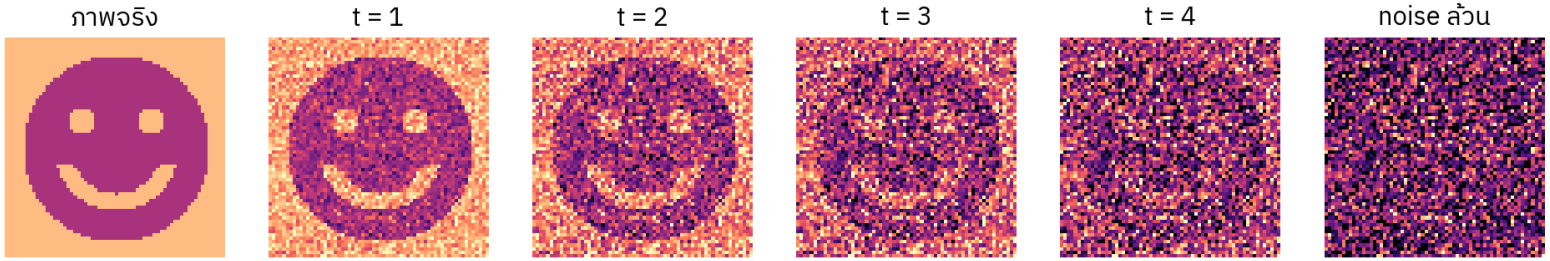

- Encoder บีบข้อมูลลง latent space
- Decoder ขยายกลับเป็นข้อมูล
- **VAE เก็บ latent เป็น "ช่วง" (μ, σ) จึงสุ่มได้**
- ได้ผลลัพธ์หลากหลายจาก input เดียว
- ใช้ในงานศิลปะ ออกแบบ

# GAN (Generative Adversarial Network)




- Generator: สร้างของปลอมจาก noise
- Discriminator: แยกของจริงกับของปลอม และให้คะแนน
- แข่งกันจน generator เก่งมาก
- **เด่นด้าน: ภาพและวิดีโอที่สมจริง**

# Diffusion Model




**Forward: ใส่ noise ทีละนิด →→→**

**←←← Reverse: โมเดลเรียนรู้การ "ลบ noise" ทีละขั้น → เริ่มจาก noise ใหม่ ได้ภาพใหม่**

## หลักการ

เรียนรู้ "การลบ noise" จากภาพที่ถูกทำลาย แล้วใช้สร้างภาพใหม่จาก noise ล้วน

## ตัวอย่างการใช้

สร้างภาพจากพรอมต์ · ซ่อมภาพเก่าที่เสียหายให้คมชัด

ส่วนที่ 3

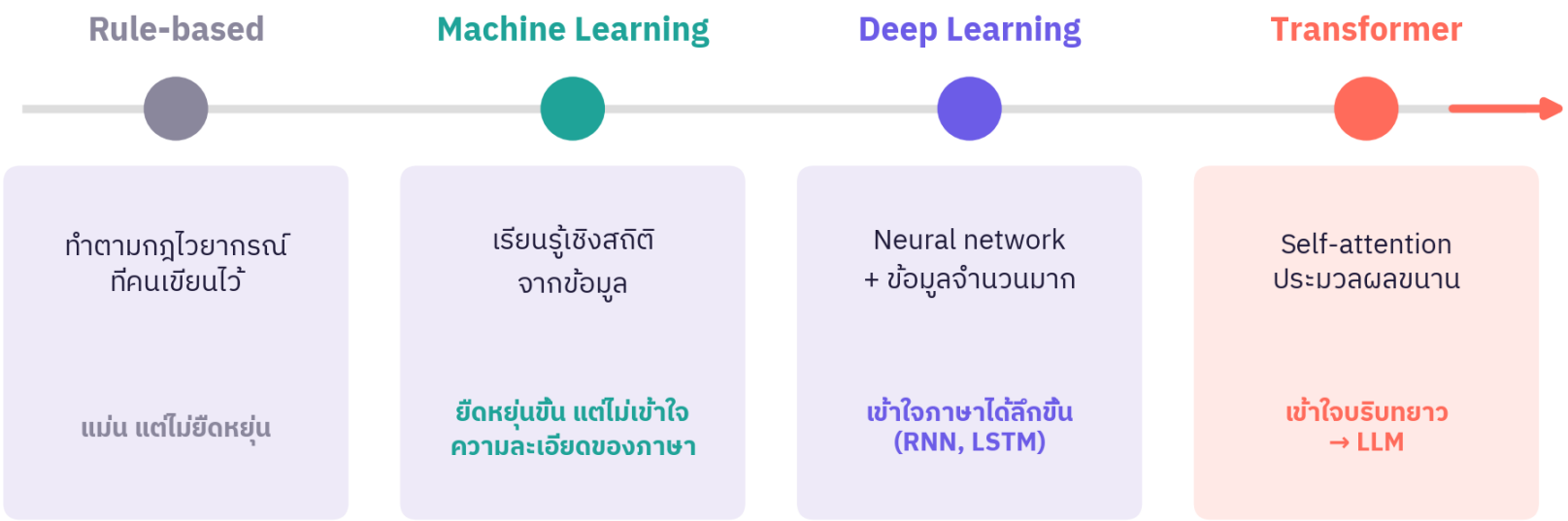
# โมเดลสร้างข้อความ: จาก RNN สู่ LLM

**จากส่วนที่แล้ว:** ภาพสร้างจาก latent space ได้ แต่ภาษาต่างออกไป: คำมี "ลำดับ" และความหมายขึ้นกับ "บริบท" จึงต้องใช้สถาปัตยกรรมอีกสายหนึ่ง

**คำถามของส่วนนี้**

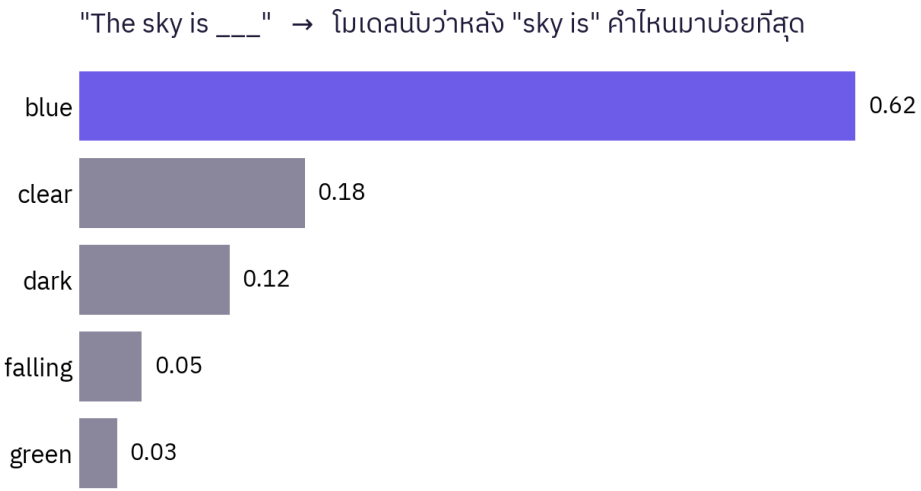
ธนาคารต้องการ virtual assistant ที่คุยกับลูกค้าเป็นภาษาธรรมชาติ — ต้อง "เข้าใจคำถาม · ตอบเป็นธรรมชาติ · จำบริบท · ไม่ให้ข้อมูลผิด" โมเดลแบบไหนทำได้?

# วิวัฒนาการของ NLP




สไลด์ถัดไป: ไล่ดูทีละรุ่น N-gram → RNN → LSTM → Attention → Transformer ว่าแต่ละรุ่นแก้ปัญหาอะไร

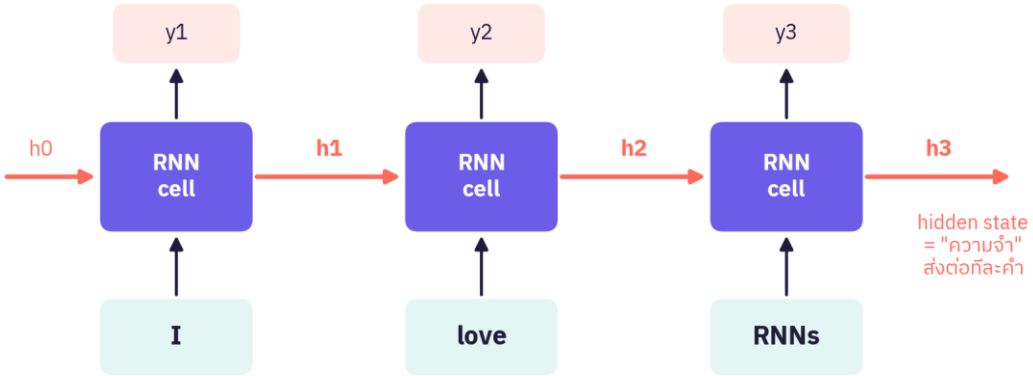
# จุดเริ่มต้น: N-gram Language Model




- นับสถิติว่าคำไหนมักตามหลังคำไหน
- ทำนายคำถัดไปจาก N-1 คำก่อนหน้า
- **ข้อจำกัด: มองบริบทได้สั้นมาก**
- ไม่เข้าใจความหมาย แค่นับความถี่

N-gram มองย้อนได้แค่ N-1 คำ ถ้าบริบทสำคัญอยู่ไกลกว่านั้น โมเดลจะมองไม่เห็น

# RNN: Neural Network ที่มี "ความจำ"

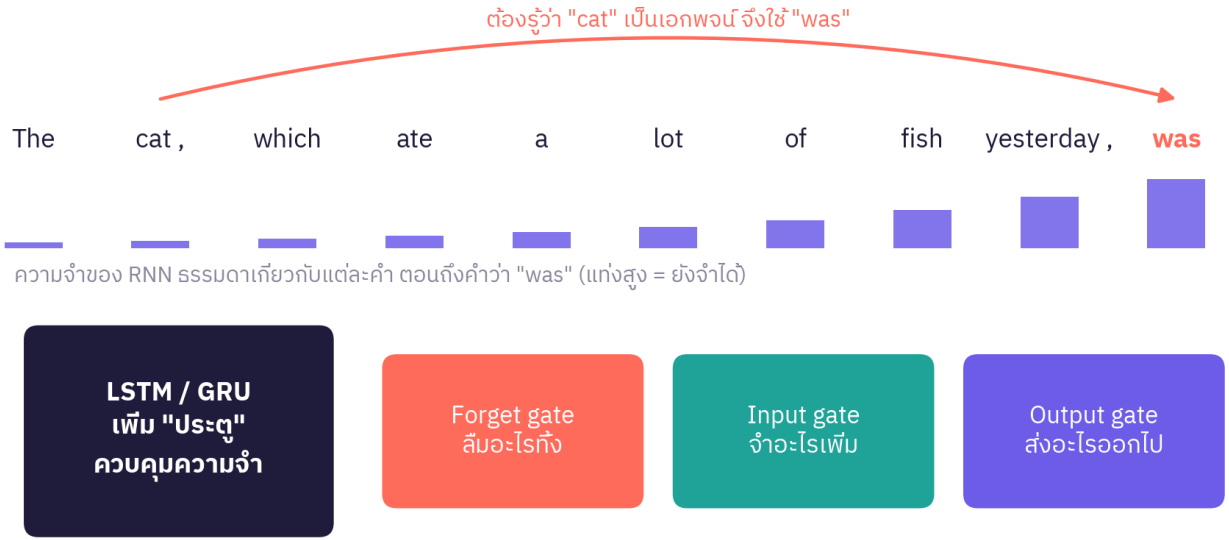



- มี loop ส่ง hidden state ต่อไปทุกขั้น
- hidden state = สรุปทุกอย่างที่อ่านมาแล้ว
- เหมาะกับข้อมูลที่มีลำดับ

**ใช้กับ**

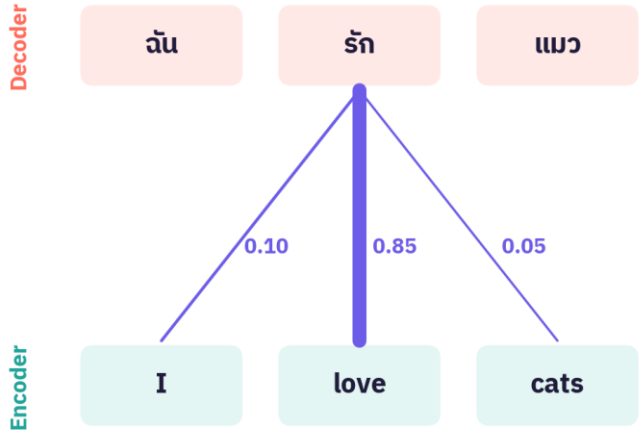
NLP · แปลภาษา · รู้จำเสียงพูด · บรรยายภาพ (image captioning) · ข้อมูล time series

# ปัญหาของ RNN → LSTM / GRU




LSTM/GRU จำได้ไกลขึ้น แต่ยังต้องอ่าน "ทีละคำ" → เทรนช้า ขยายขนาดยาก

# Seq2seq + Attention

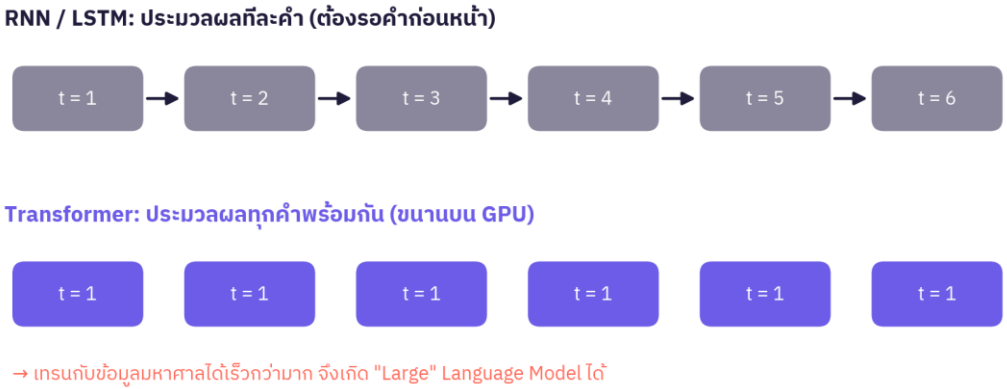



ตอนเขียนคำว่า "รัก"
decoder โฟกัส (attention)
ไปที "love" มากที่สุด

เส้นหนา = น้ำหนักมาก

- Encoder อ่านประโยคต้นทาง
- Decoder เขียนประโยคปลายทางทีละคำ
- **Attention: ทุกครั้งที่เขียน ให้ "หันกลับไปดู" คำต้นทางที่เกี่ยวข้อง**
- ทำให้การแปลภาษาแม่นขึ้นมาก

# Transformer (2017): Attention Is All You Need




## ความคิดหลัก

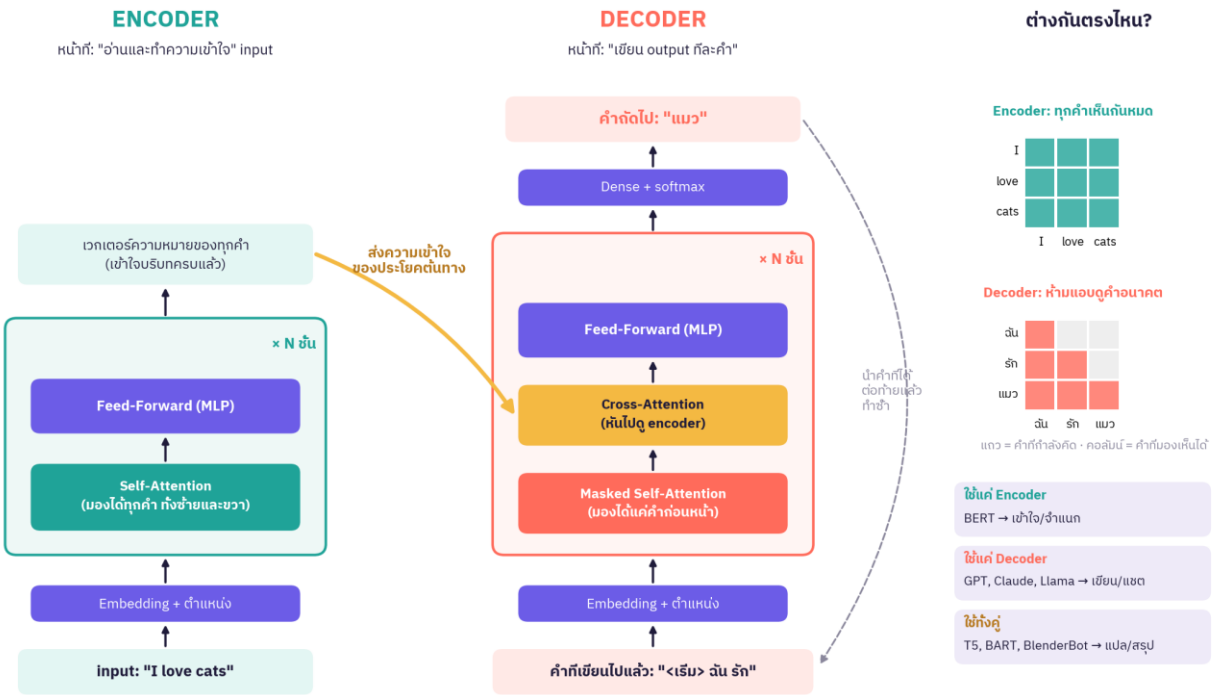
ตัด RNN ทิ้ง ใช้ attention อย่างเดียว

## ผลลัพธ์

ประมวลผลขนานบน GPU → เทรนกับข้อมูลมหาศาลได้

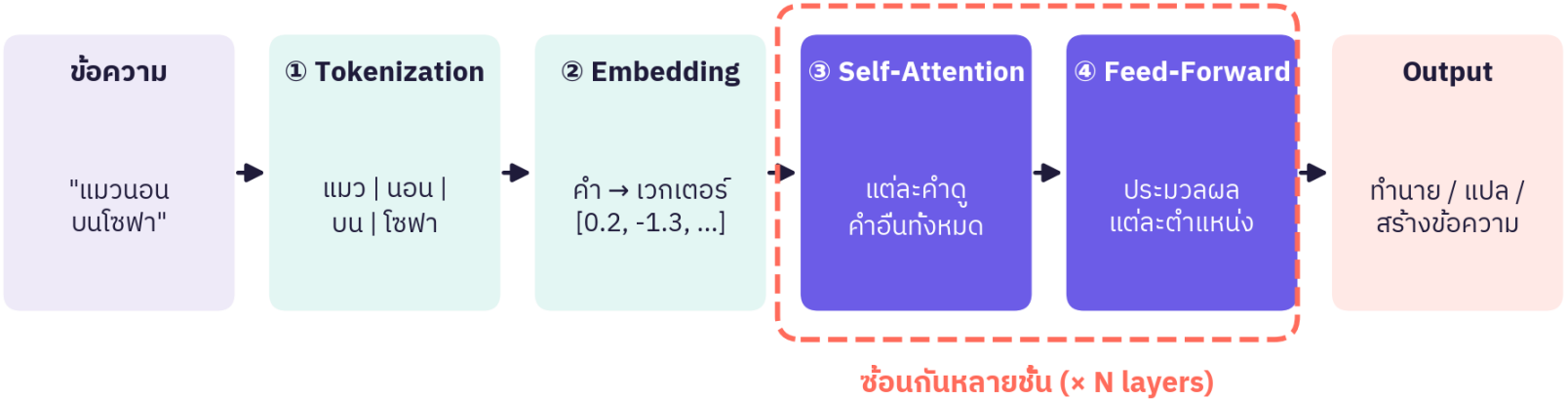
Vaswani et al., 2017 — พื้นฐานของ GPT, BERT, T5 และ LLM แทบทุกตัวในปัจจุบัน

# Transformer ต้นฉบับ: Encoder + Decoder



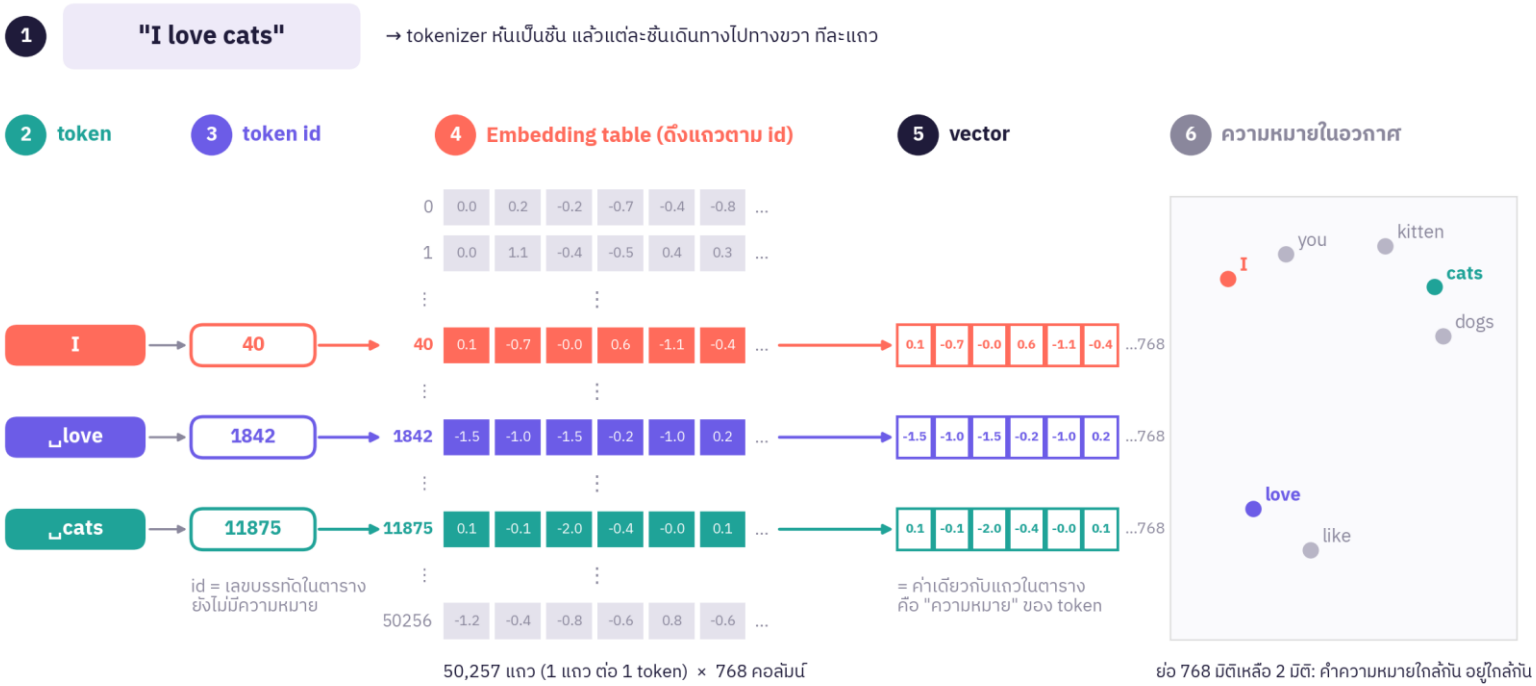

โครงนี้มาจากบทความ "Attention Is All You Need" (2017) ซึ่งใช้แปลภาษา · GPT ตัด encoder และ cross-attention ออก เหลือ decoder ล้วน

# ข้างใน Transformer ทำอะไรบ้าง




บ่ายนี้เราจะได้เห็นขั้น ① Tokenization และ ② Embedding ด้วยตาตัวเองใน Colab

# ขั้น ① ②: จาก Token สู่ Vector

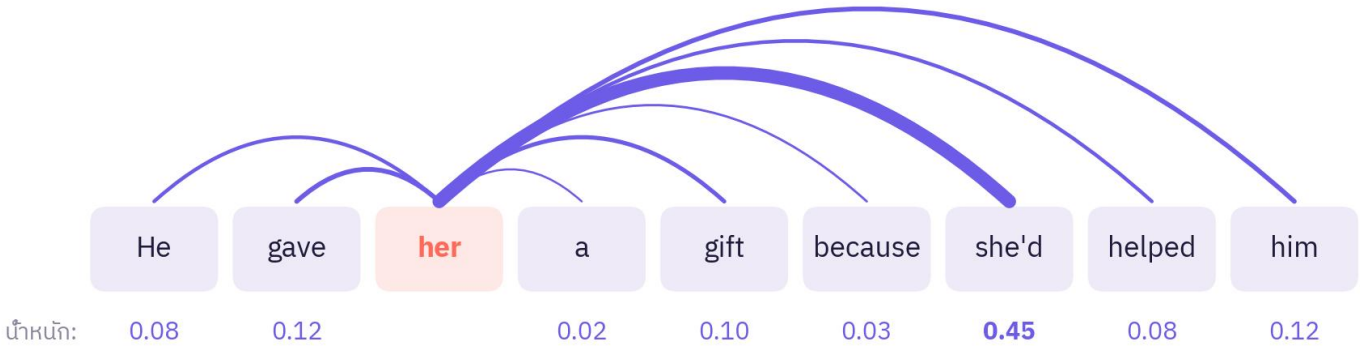



token id และตัวเลขในตารางเป็นตัวอย่างเพื่ออธิบาย · ตัวเลขในตารางจริงได้มาจากการเทรน (ปรับด้วย backpropagation เหมือน weight ทั่วไป)

# Self-Attention: แต่ละคำ "มองหา" คำที่เกี่ยวข้อง

คำว่า "her" หมายถึงใคร? Self-attention ให้ "her" ให้น้ำหนักกับทุกคำในประโยค

→ น้ำหนักสูงสุดอยู่ที่ "she'd" ⇒ โมเดลรู้ว่าเป็นคนเดียวกัน (ตัวเลขเป็นตัวอย่างเพื่ออธิบาย)



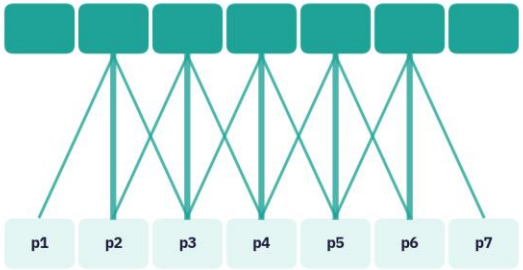

ทุกคำทำแบบนี้พร้อมกัน และทำซ้ำหลายชั้น → โมเดลเข้าใจความสัมพันธ์ในประโยคได้ลึกขึ้นเรื่อยๆ

# Attention ต่างจาก CNN / Dense อย่างไร

### Convolution (AlexNet / CNN)

น้ำหนักชุดเดียว [0.2, 0.6, 0.2] ใช้ซ้ำทุกตำแหน่ง ทุกภาพ

มองแค่ "เพื่อนบ้าน" ในหน้าต่างเล็กๆ (เช่น 3×3)

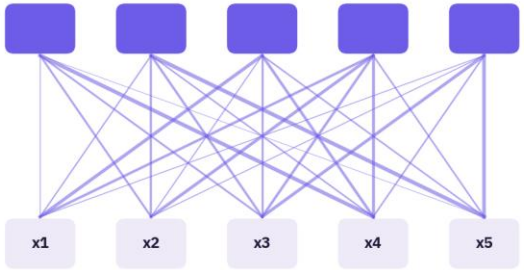



เรียนตอนเทรน แล้ว "ตายตัว" ตอนใช้งาน

### Dense / Fully Connected

แต่น้ำหนักทุกเส้นก็ "ตายตัว" หลังเทรน

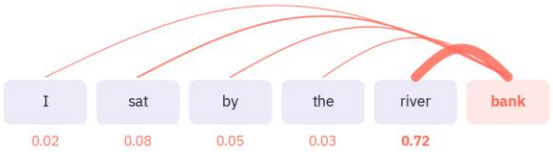
ทุก input ต่อกับทุก output (มองได้ทั้งหมด)




ข้อมูลเข้าต่างกัน → ใช้น้ำหนักชุดเดิม

### Self-Attention (Transformer)

น้ำหนัก "คำนวณใหม่ทุกครั้ง" จากประโยคที่เข้ามา



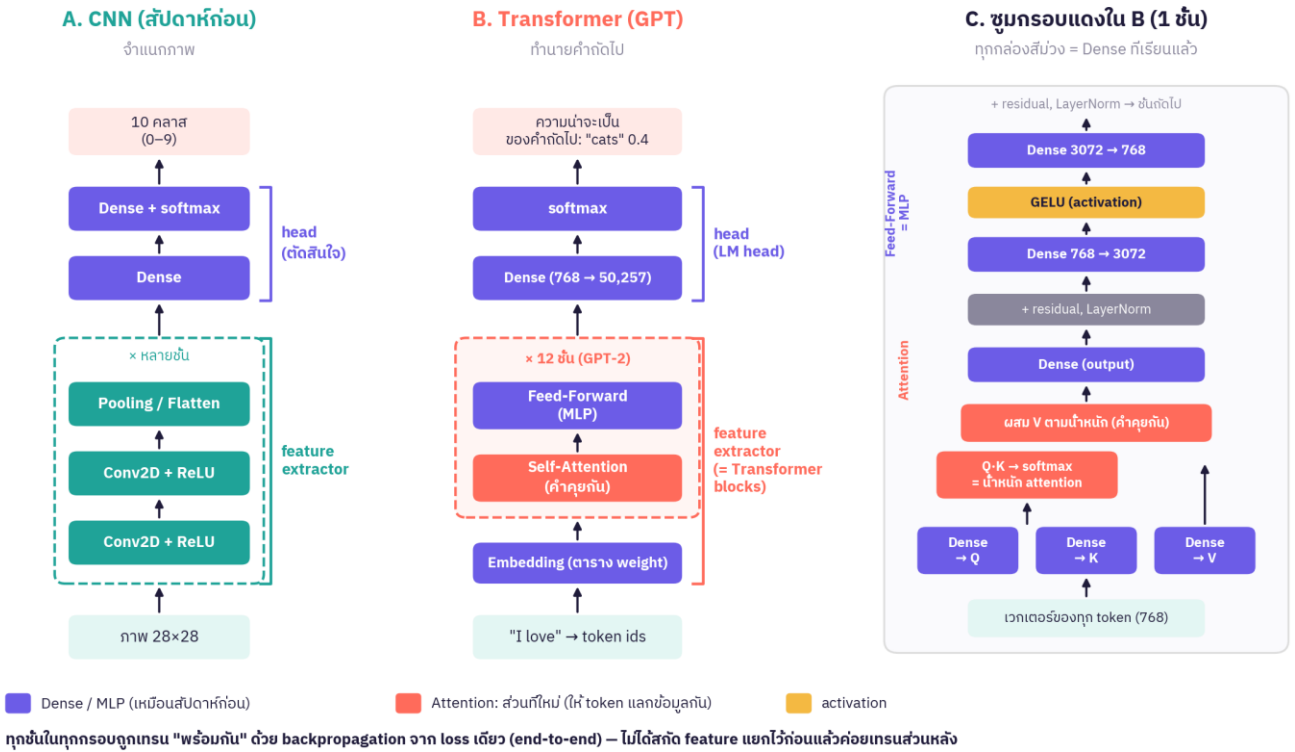

ประโยค 1: "bank" มอง "river" → ตลิ่ง




ประโยค 2: "bank" มอง "cash" → ธนาคาร

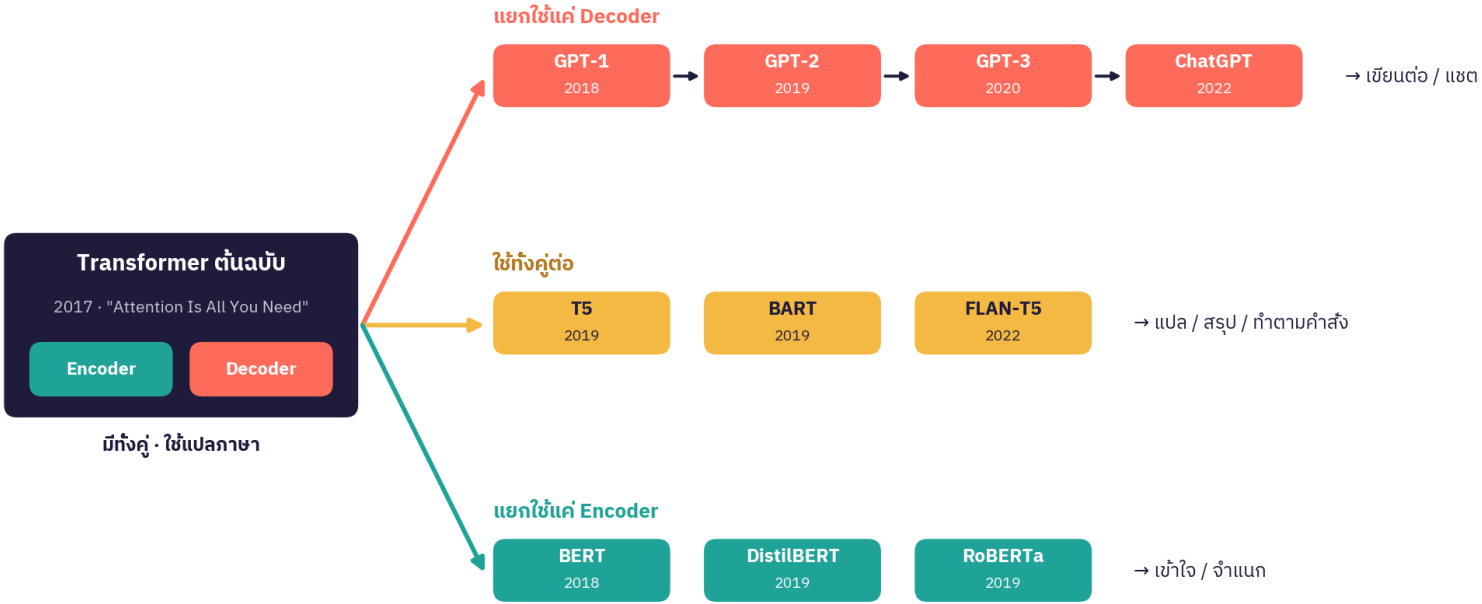
มองได้ทุกคำ + เลือกเองว่าจะฟังใคร (ตัวเลขเป็นตัวอย่าง)

# Transformer ก็คือ Neural Network



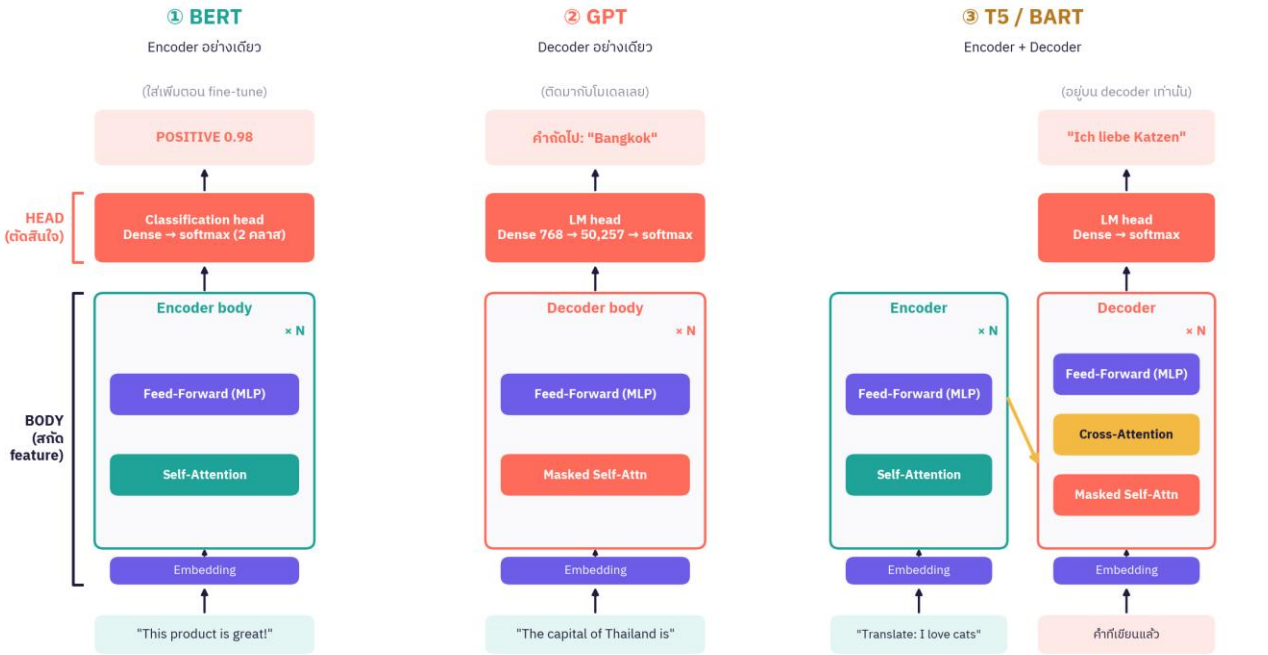

**ทุกชั้นในทุกกรอบถูกเทรน "พร้อมกัน" ด้วย backpropagation จาก loss เดียว (end-to-end) — ไม่ได้สกัด feature แยกไว้ก่อนแล้วค่อยเทรนส่วนหลัง**

# จาก Transformer ต้นฉบับ แตกเป็น 3 สาย




ในแล็บ: GPT-2 = สายบน · FLAN-T5, BlenderBot = สายกลาง · DistilBERT, BERT-NER (Lab 5) = สายล่าง · แถวกลางและล่างเรียงตามปี ไม่ได้พัฒนาต่อกันทุกตัว

# Body + Head ของแต่ละสาย




⚠ **อย่าสับสน: "head" ในภาพนี้ = ชั้นสุดท้ายที่แปลงเวกเตอร์เป็นคำตอบ (เหมือน Dense + softmax ของ CNN)**

ส่วน "multi-head attention" คือ attention หลายชุดทำงานคู่ขนานภายในชั้น attention (เช่น GPT-2 มี 12 heads ต่อชั้น) — คนละเรื่องกัน

# ตัวอย่าง: GPT

**G**

**Generative**
สร้างข้อความใหม่ได้

**P**

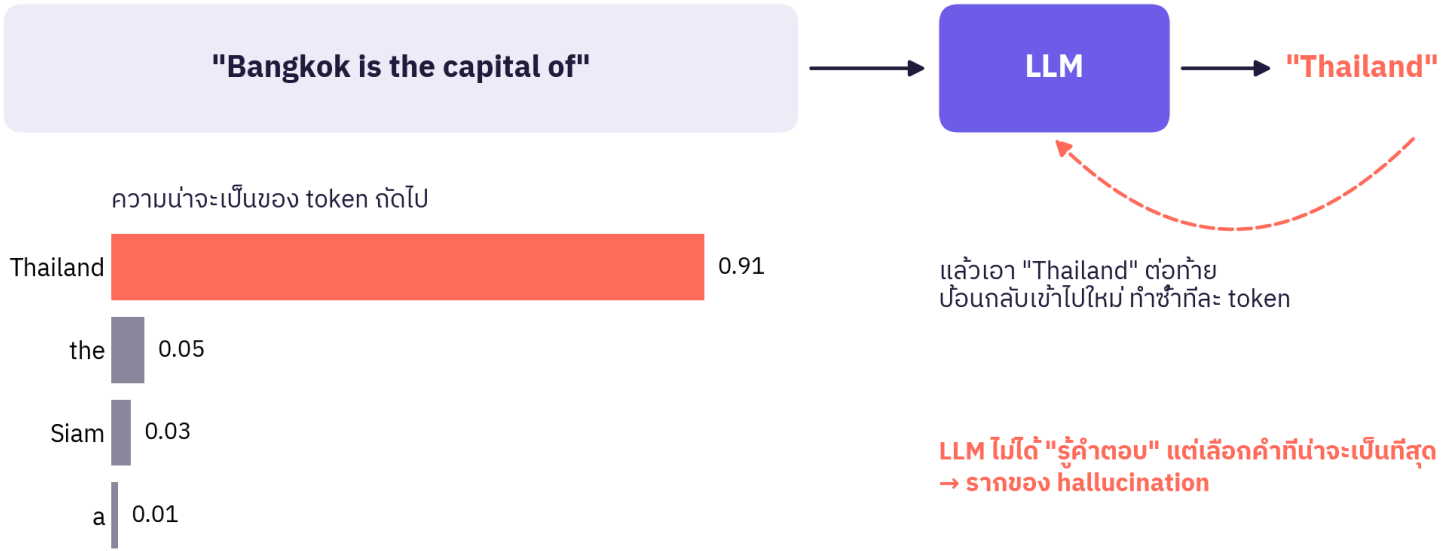
**Pre-trained**
ฝึกล่วงหน้าด้วยข้อความมหาศาล

**T**

**Transformer**
ใช้สถาปัตยกรรม Transformer (decoder)

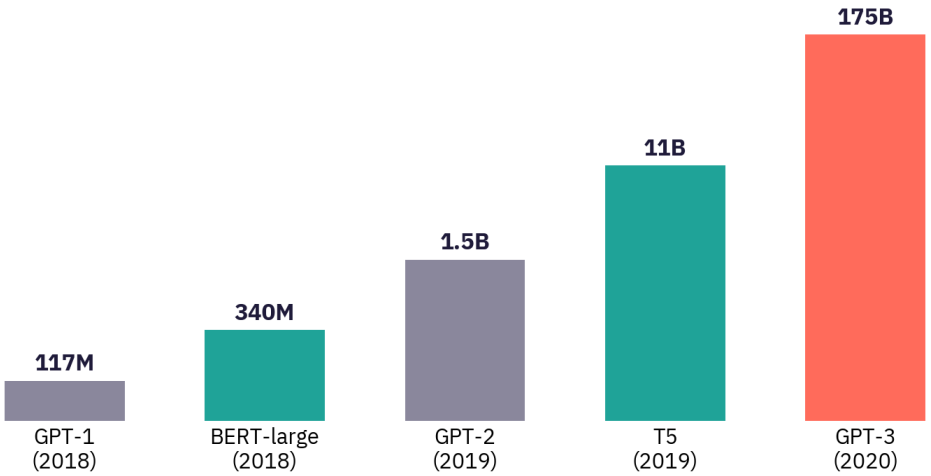
GPT เรียนรู้รูปแบบจากข้อมูล แล้วทำนาย/สร้างข้อความต่อ และ fine-tune ไปใช้งานอื่นได้หลากหลาย

# Decoder สร้างข้อความทีละ Token



# Large Language Model (LLM) คืออะไร

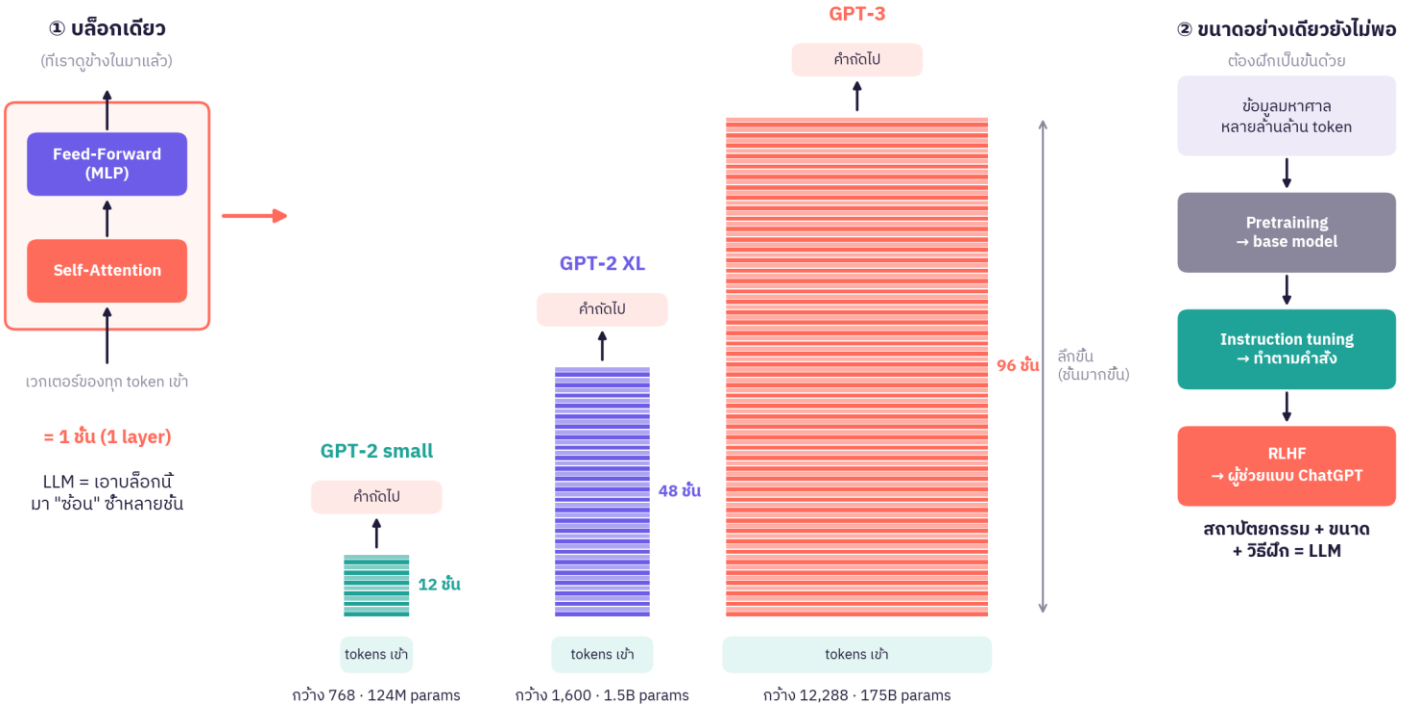
จำนวนพารามิเตอร์ (สเกล log) — ภายใน 2 ปี โตขึ้นกว่า 1,000 เท่า




M = ล้าน, B = พันล้านพารามิเตอร์ · โมเดลรุ่นใหม่ส่วนใหญ่ไม่เปิดเผยตัวเลขนี้แล้ว

- Foundation model ฝึกด้วยข้อมูลมหาศาล (เว็บไซต์ หนังสือ) ระดับ petabyte
- มีพารามิเตอร์หลักพันล้าน
- พารามิเตอร์ = ค่าที่กำหนดพฤติกรรมโมเดล เช่น น้ำหนักของคำ "happy" / "sad"
- **เก่งที่สุดในการทำนายคำถัดไป**

# LLM = บล็อกเดิมซ้อนซ้ำหลายชั้น



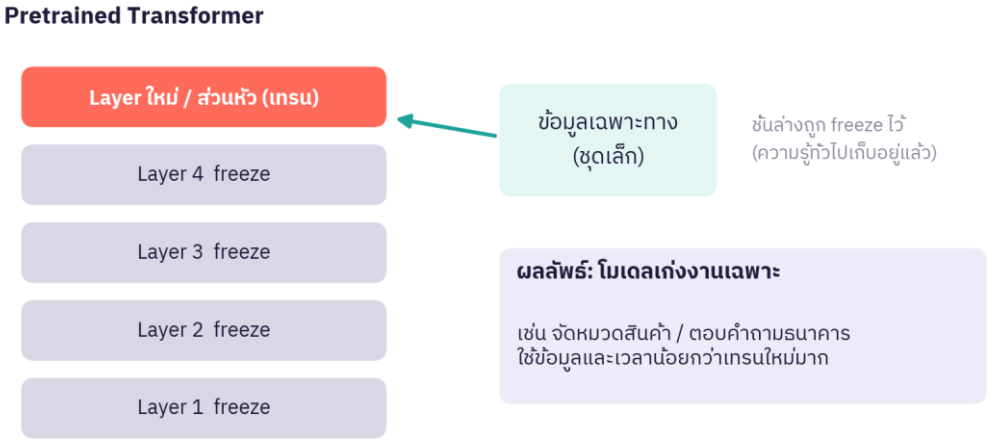

ทุกชั้นมีโครงเหมือนกัน แต่ weight ของแต่ละชั้นต่างกัน (เรียนแยกกัน) · ความกว้างในภาพย่อส่วนเพื่อให้ดูง่าย · ตัวเลขเป็นค่าที่เปิดเผยของ GPT-2 และ GPT-3

# ข้อมูลเดียวกัน: โมเดลเล็ก vs โมเดลใหญ่




ข้อมูลเข้าเหมือนกัน · ตัวเลขความน่าจะเป็นและข้อความเป็นตัวอย่างเพื่ออธิบาย · หน้าที่ของชั้นที่ระบุเป็นภาพโดยประมาณ

# Fine-tuning: ปรับโมเดลสำเร็จรูปให้เก่งงานเฉพาะ




## Fine-tune RNN

อาจต้องปรับ weight และโครงสร้างเครือข่ายให้เข้ากับงาน

## Fine-tune Transformer

ส่วนใหญ่ freeze ไว้ เทรนแค่ชั้นท้ายๆ ของงานนั้น

เชื่อมกับ Transfer Learning ที่เรียนสัปดาห์ก่อน — หลักการเดียวกัน

# Pretrain ครั้งเดียว → Fine-tune ได้หลายงาน

**Pretrained LLM**

เรียนรู้ภาษาทั่วไป เช่น จัดหมวดข้อความทั่วไป

→

**+ ข้อมูลชุดเล็ก**

คำอธิบายสินค้าของร้านค้าปลีก

→

**โมเดลเฉพาะทาง**

จัดสินค้าเป็นหมวด: อิเล็กทรอนิกส์ / เสื้อผ้า

**ข้อควรระวัง**

LLM เขียนได้น่าเชื่อถือในทุกเรื่อง แต่อาจ "ผิดโดยฟังดูถูก" มี bias และส่งผลต่อสังคม → หัวข้อถัดไป

# Base model vs Instruction model

| | Base model | Instruction model |
|---|---|---|
| ฝึกด้วย | ข้อความทั่วไป (pretraining) | base + ตัวอย่าง "คำสั่ง → คำตอบที่ดี" |
| มองข้อความเราว่า | ต้นเรื่องที่ต้องเขียนต่อ | คำสั่งที่ต้องทำให้เสร็จ |
| "Translate to German: I love cats." | → "Translate to French: ..." (เขียนต่อ) | → "Ich liebe Katzen." |
| ชื่อบน Hugging Face | Qwen2.5-0.5B | Qwen2.5-0.5B-Instruct |
| ในแล็บบ่ายนี้ | GPT-2 | FLAN-T5 |

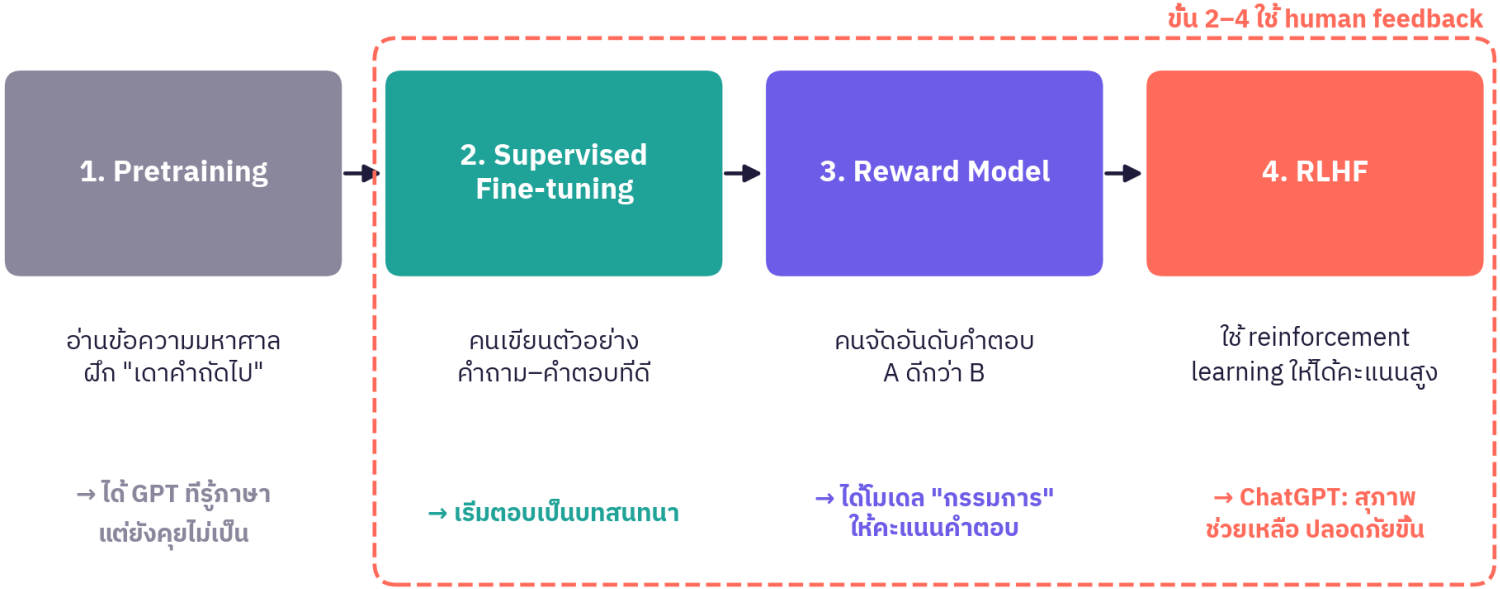
base = รู้เยอะแต่ไม่รู้ว่าเราต้องการอะไร · instruction = รู้เยอะและรู้ว่าต้องทำงานให้เรา

# GPT vs ChatGPT

| ประเด็น | GPT | ChatGPT |
|---|---|---|
| เป้าหมาย | สร้างข้อความหลากหลายรูปแบบ | สร้าง "บทสนทนา" |
| วิธีเทรน | Supervised learning เป็นหลัก | Supervised + Reinforcement learning |
| Feedback จากมนุษย์ | ไม่ใช้ในการเรียนรู้ | ใช้ RLHF (Reinforcement Learning from Human Feedback) |

RLHF: ให้คนให้คะแนนคำตอบ → สร้าง reward model → ใช้ปรับโมเดลให้ตอบแบบที่คนชอบ

# ChatGPT ถูกสร้างขึ้นอย่างไร




ขั้น 4 คือ Reinforcement Learning: โมเดลเรียนจาก "รางวัล" ที่ได้จากกรรมการ (reward model) — GenAI ใช้ RL แบบนี้ปรับพฤติกรรม

สรุปส่วนที่ 2–3

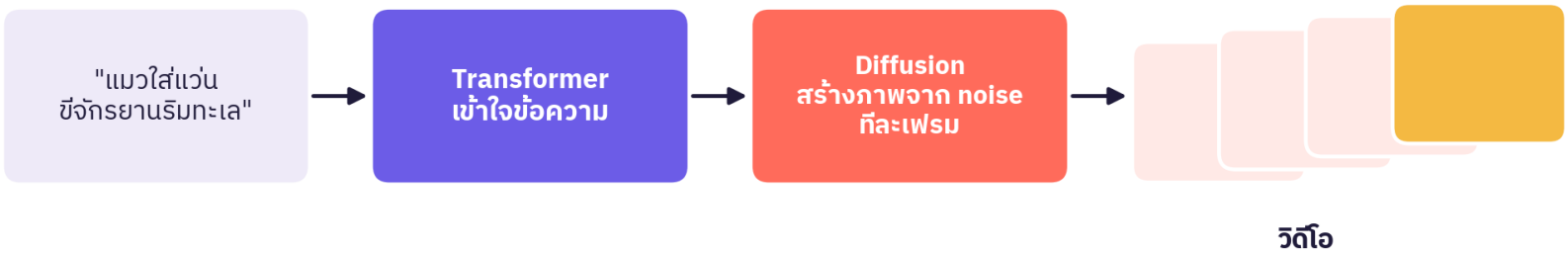
# เปรียบเทียบ 5 สถาปัตยกรรม

| โมเดล | วิธีเทรน | จุดเด่น | งานที่เหมาะ |
|---|---|---|---|
| **RNN** | Loop ส่ง hidden state | ข้อมูลที่มีลำดับ | แปลภาษา, เสียง, time series |
| **Transformer** | Self-attention | บริบทยาว, เทรนขนาน | LLM, แปลภาษา, chatbot |
| **GAN** | แข่งขัน G vs D | ภาพคมและสมจริง | สร้างภาพ/วิดีโอ |
| **VAE** | Encoder–decoder + distribution | สุ่มได้หลากหลาย | ศิลปะ, ออกแบบ |
| **Diffusion** | เรียนลบ noise (สถิติ) | คุณภาพสูง, ตามพรอมต์ | Text-to-image, ซ่อมภาพ |

สรุปส่วนที่ 2–3

# เฉลยโจทย์: สร้างวิดีโอจากข้อความ

**คำตอบ: ใช้หลายสถาปัตยกรรมร่วมกัน (โมเดลวิดีโอปัจจุบันจำนวนมากเป็น "Diffusion Transformer")**




**ย้อนดูสิ่งที่เรียน**

Transformer (ส่วนที่ 3) เข้าใจข้อความ + Diffusion (ส่วนที่ 2) สร้างภาพ → รวมกันเป็นโมเดลวิดีโอ

ส่วนที่ 4

# AI Hallucination

**จากส่วนที่แล้ว:** LLM เลือกคำที่ "น่าจะเป็นที่สุด" ทีละ token ตามรูปแบบที่เรียนมา — ไม่ได้ค้นหาความจริง

**คำถามของส่วนนี้**

ถ้า LLM ไม่รู้คำตอบ มันจะทำอย่างไร? และเราจะป้องกันได้อย่างไร?

# AI Hallucination คืออะไร

## นิยาม

โมเดลสร้างคำตอบที่นำเสนอว่า "ถูกต้อง" แต่มนุษย์มองว่า ไม่สมจริง ผิด ไม่เกี่ยวข้อง หรือไร้สาระ

คล้าย "ภาพหลอน" ของมนุษย์

## กรณีจริง

ChatGPT อ้างว่านายกเทศมนตรีคนหนึ่งในออสเตรเลีย "ถูกตัดสินจำคุกคดีสินบน"

ความจริง: เขาเป็นคนแจ้งเบาะแสคดีสินบนต่อเจ้าหน้าที่

ที่มา: Reuters

# สาเหตุของ Hallucination

### Bias ในข้อมูลฝึก
ข้อมูลเอียง → คำตอบเอียง

### เทรนไม่พอ
ข้อมูลน้อยหรือคุณภาพต่ำ

### โมเดลซับซ้อน
ยากต่อการควบคุมและตรวจสอบ

### ขาดคนตรวจ
ไม่มี human oversight

### รากของปัญหา (จำสไลด์ "Decoder สร้างข้อความทีละ token" ได้ไหม?)
LLM ไม่ได้ "เข้าใจ" ความหมาย แต่เลือกคำที่น่าจะเป็นที่สุดจากรูปแบบทางสถิติ → ประโยคจึงลื่นไหลแม้เนื้อหาผิด

# Hallucination สร้างปัญหาอะไรได้บ้าง

1

## ข้อมูลผิด

สรุปเอกสารกฎหมายผิด → ข้อพิพาท/ฟ้องร้อง

2

## มุมมองเอียง

สร้างความเห็นที่มี bias หรือชี้นำผิด

3

## ระบบที่อ่อนไหว

ส่งข้อมูลผิดให้ระบบการแพทย์หรือรถยนต์ไร้คนขับ

กลับไปที่โจทย์สรุปผลตรวจคนไข้ตอนเปิดคาบ: ตอนนี้คุณจะโหวตข้อไหน?

# ป้องกันและลด Hallucination

## ฝั่งผู้พัฒนาโมเดล

- ลด bias และฝึกด้วยข้อมูลคุณภาพสูง
- ป้องกันการบิดเบือน input
- ประเมินและปรับปรุงต่อเนื่อง
- Fine-tune ด้วยข้อมูลเฉพาะทาง

## ฝั่งผู้ใช้งาน

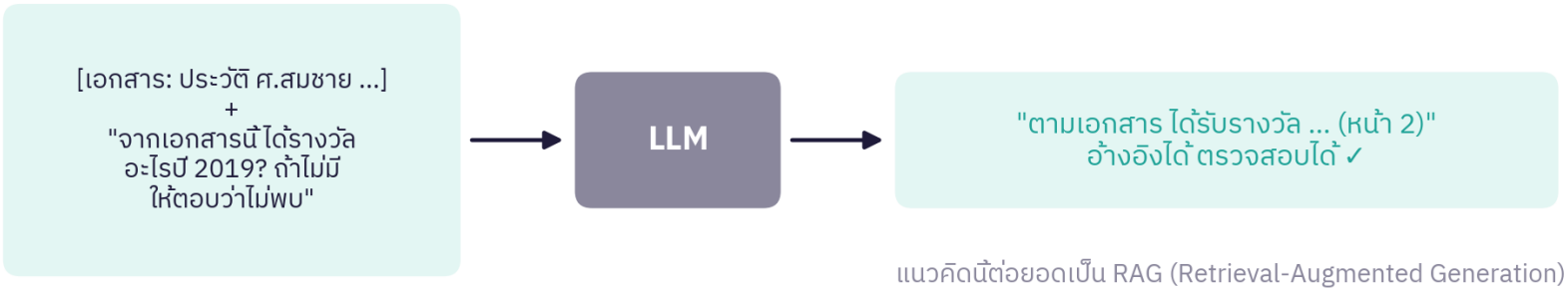
- ระวังเสมอ: โมเดลเดาคำ ไม่ได้เข้าใจ
- ให้มนุษย์ตรวจสอบข้อเท็จจริง
- ใส่บริบท (context) เพิ่มในพรอมต์
- ทดสอบซ้ำอย่างต่อเนื่อง

# พลังของ Context: ถามแบบไหนได้คำตอบดีกว่า

### ถามแบบไม่มีบริบท




### ถามแบบใส่บริบท (context) ในพรอมต์




แนวคิดนี้ต่อยอดเป็น RAG (Retrieval-Augmented Generation)

# Libraries และเครื่องมือสำหรับ Generative AI

| | |
|---|---|
| **แอปพลิเคชัน** | Chatbot, ระบบสรุปเอกสาร, ผู้ช่วย AI |
| **LangChain + Pydantic** | ต่อ LLM กับพรอมต์/ข้อมูล · ตรวจรูปแบบข้อมูล |
| **Hugging Face** | Model Hub · Transformers · Datasets · Tokenizers |
| **PyTorch \| TensorFlow / Keras** | Framework สำหรับสร้างและเทรน neural network |
| **GPU / Google Colab** | ฮาร์ดแวร์และสภาพแวดล้อมที่ใช้รัน |

PyTorch เด่นเรื่องความยืดหยุ่น (นิยมในงานวิจัย) · TensorFlow/Keras เด่นเรื่องขยายสู่ production · สัปดาห์ก่อนเราใช้ Keras

# Hugging Face และสิ่งที่เราจะใช้บ่ายนี้

### Model Hub
คลังโมเดล pretrained หลายแสนตัว

### Transformers
โหลดและใช้โมเดลด้วยโค้ดไม่กี่บรรทัด

### Datasets
ชุดข้อมูลพร้อมใช้หลายภาษา

### Tokenizers
ตัดคำให้โมเดลแบบรวดเร็ว

### บ่ายนี้ใน Colab
- AutoTokenizer
- AutoModel + generate()
- BlenderBot, FLAN-T5, GPT-2
- Keras สร้าง VAE

สรุป

# สรุปและคำถามทบทวน

- ① GenAI เรียนรู้การกระจายของข้อมูล แล้วสร้างข้อมูลใหม่
- ② ภาพ: สุ่มจุดใน latent space → VAE / GAN / Diffusion
- ③ ข้อความ: RNN → Attention → Transformer = NN ที่ให้คำคุยกัน → LLM = ซ้อนหลายชั้น + ฝึกเป็นขั้น (instruction, RLHF)
- ④ Hallucination: เลือกคำตามสถิติ → ใช้ context และคนตรวจ
- ⑤ บ่ายนี้: ลงมือใน Colab

**คำถามทบทวน**

1. GAN ต่างจาก VAE อย่างไร?

2. ทำไม Transformer เทรนเร็วกว่า RNN?

3. Hallucination เกิดจากอะไร และลดได้อย่างไร?